

Міністерство освіти і науки України
Національний Університет “Львівська Політехніка”



Лабораторна робота
з дисципліни
“Об'єктно-орієнтоване програмування, частина 2”

Виконав:
студент гр. ІХ-22
Пристайко М.

Прийняв:
Гордійчук-Бублівська О.В

Лабораторна робота 5.

Тема: Асинхронне програмування в телекомунікаційних системах: взаємодія з базою даних через SQLAlchemy.

Мета: Ознайомитися з принципами асинхронного програмування в Python за допомогою модуля `asyncio`, навчитися працювати з асинхронними запитами до бази даних через SQLAlchemy, реалізувати систему збору та обробки даних про мережеві вузли.

Завдання:

```
import asyncio
import random
from datetime import datetime
from sqlalchemy import Column, Integer, String, Float, DateTime, text
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker, declarative_base
from sqlalchemy.future import select
```

```
DATABASE_URL = "sqlite+aiosqlite:///network.db"
Base = declarative_base()
engine = create_async_engine(DATABASE_URL, echo=False)
AsyncSessionLocal = sessionmaker(engine, class_=AsyncSession,
expire_on_commit=False)
```

```
class Node(Base):
    __tablename__ = "nodes"

    id = Column(Integer, primary_key=True, autoincrement=True)
    ip_address = Column(String(15), unique=True, nullable=False)
    hostname = Column(String(50), nullable=False)
    status = Column(String(20), default="unknown")
    response_ms = Column(Float, default=0.0)
    last_checked = Column(DateTime, default=datetime.now)
```

```
class DatabaseManager:
    @staticmethod
    async def create_tables():
        async with engine.begin() as conn:
            await conn.run_sync(Base.metadata.create_all)

    @staticmethod
    async def reset_nodes():
        async with AsyncSessionLocal() as session:
            await session.execute(text("DELETE FROM nodes"))
            await session.commit()
```

```
class NodeRepository:
    @staticmethod
```

```
async def add_nodes(nodes_data: list[dict]):
    async with AsyncSessionLocal() as session:
        nodes = [Node(**n) for n in nodes_data]
        session.add_all(nodes)
        await session.commit()
```

@staticmethod

```
async def get_all_nodes():
    async with AsyncSessionLocal() as session:
        result = await session.execute(select(Node))
        return result.scalars().all()
```

@staticmethod

```
async def update_node_status(node_id: int, status: str, response_ms: float):
    async with AsyncSessionLocal() as session:
        result = await session.execute(select(Node).where(Node.id == node_id))
        node = result.scalar_one_or_none()
        if node:
            node.status = status
            node.response_ms = response_ms
            node.last_checked = datetime.now()
            await session.commit()
```

class NetworkMonitor:

@staticmethod

```
async def _ping_node(node: Node):
    # Симуляція затримки мережі
    delay = random.uniform(0.01, 0.3)
    await asyncio.sleep(delay)
    response_ms = round(delay * 1000, 1)

    # Логіка визначення статусу за IP та затримкою
    last_octet = int(node.ip_address.split('.')[-1])
    if last_octet in (13, 14, 15):
        status = "offline"
    elif response_ms > 250:
        status = "degraded"
    else:
        status = "active"

    return {"id": node.id, "status": status, "response_ms": response_ms}
```

```

@staticmethod
async def monitor_all_nodes():
    nodes = await NodeRepository.get_all_nodes()

    results = await asyncio.gather(*(NetworkMonitor._ping_node(n) for n in
nodes))

    for r in results:
        await NodeRepository.update_node_status(r["id"], r["status"],
r["response_ms"])
    return results

class ReportPrinter:
    @staticmethod
    async def print_nodes(title="Список вузлів"):
        nodes = await NodeRepository.get_all_nodes()
        print(f"\n{'='*70}\n📊 {title}\n{'='*70}")
        print(f"{'ID':<5} {'IP':<16} {'Host':<20} {'Status':<10} {'Response(ms)':>12}")
        print("-" * 70)

        for n in nodes:
            icon = {"active": "✅", "offline": "🚫", "degraded": "⚠️", "unknown":
"❓"}.get(n.status, "❓")
            print(f"{'ID':<5} {'IP':<16} {'Host':<20} {icon}
{n.status:<10} {n.response_ms:>10.1f}")
            print("=" * 70)

    @staticmethod
    async def print_statistics():
        nodes = await NodeRepository.get_all_nodes()
        total = len(nodes)
        print(f"\n📊 Статистика мережі")

        for status, icon in [("active", "✅"), ("offline", "🚫"), ("degraded", "⚠️")]:
            filtered = [n for n in nodes if n.status == status]
            avg = sum(n.response_ms for n in filtered) / len(filtered) if filtered else 0
            print(f"{icon} {status.upper():<10}: {len(filtered):>2} вузлів (середній
відгук: {avg:.1f} мс)")

        active = len([n for n in nodes if n.status == "active"])

```

```
availability = (active / total * 100) if total else 0
print(f"\n💡 Доступність мережі: {availability:.1f}%")
```

```
NODES_DATA = [{"ip_address": f"192.168.1.{i}", "hostname": f"device-{i}"} for
i in range(1, 16)]
```

```
async def main():
    await DatabaseManager.create_tables()
    await DatabaseManager.reset_nodes()
    await NodeRepository.add_nodes(NODES_DATA)

    await ReportPrinter.print_nodes("Вузли ДО моніторингу")
    await NetworkMonitor.monitor_all_nodes()
    await ReportPrinter.print_nodes("Вузли ПІСЛЯ моніторингу")
    await ReportPrinter.print_statistics()
```

```
# Якщо запуск у середовищі з уже існуючим event loop (наприклад,
Jupyter/Colab)
```

```
import nest_asyncio
nest_asyncio.apply()
```

```
if __name__ == "__main__":
    asyncio.run(main())
```

Результат:

...

Вузли ДО моніторингу

ID	IP	Host	Status	Response(ms)
1	192.168.1.1	device-1	unknown	0.0
2	192.168.1.2	device-2	unknown	0.0
3	192.168.1.3	device-3	unknown	0.0
4	192.168.1.4	device-4	unknown	0.0
5	192.168.1.5	device-5	unknown	0.0
6	192.168.1.6	device-6	unknown	0.0
7	192.168.1.7	device-7	unknown	0.0
8	192.168.1.8	device-8	unknown	0.0
9	192.168.1.9	device-9	unknown	0.0
10	192.168.1.10	device-10	unknown	0.0
11	192.168.1.11	device-11	unknown	0.0
12	192.168.1.12	device-12	unknown	0.0
13	192.168.1.13	device-13	unknown	0.0
14	192.168.1.14	device-14	unknown	0.0
15	192.168.1.15	device-15	unknown	0.0

Вузли ПІСЛЯ моніторингу

ID	IP	Host	Status	Response(ms)
1	192.168.1.1	device-1	degraded	260.6
2	192.168.1.2	device-2	active	163.2
3	192.168.1.3	device-3	active	115.6
4	192.168.1.4	device-4	active	238.1
5	192.168.1.5	device-5	active	174.7
6	192.168.1.6	device-6	active	63.3
7	192.168.1.7	device-7	active	87.6
8	192.168.1.8	device-8	active	71.5
9	192.168.1.9	device-9	active	159.4
10	192.168.1.10	device-10	active	103.2
11	192.168.1.11	device-11	active	199.3
12	192.168.1.12	device-12	active	60.8
13	192.168.1.13	device-13	offline	106.0
14	192.168.1.14	device-14	offline	81.4
15	192.168.1.15	device-15	offline	166.1

Статистика мережі

- ACTIVE : 11 вузлів (середній відгук: 130.6 мс)
- OFFLINE : 3 вузлів (середній відгук: 117.8 мс)
- DEGRADED : 1 вузлів (середній відгук: 260.6 мс)

Доступність мережі: 73.3%

Висновок:

У ході лабораторної роботи було побудовано асинхронну модель взаємодії з реляційною базою даних для моніторингу мережевої інфраструктури. Завдяки інтеграції **asyncio** та асинхронного двигуна **SQLAlchemy**, вдалося реалізувати неблокуючу обробку запитів, що забезпечило паралельне оновлення статусів вузлів. Отримані результати підтверджують високу ефективність асинхронних підходів у розробці програмного забезпечення для телекомунікаційних систем із високою інтенсивністю даних.